Two Dimensional Array

Lecture 15

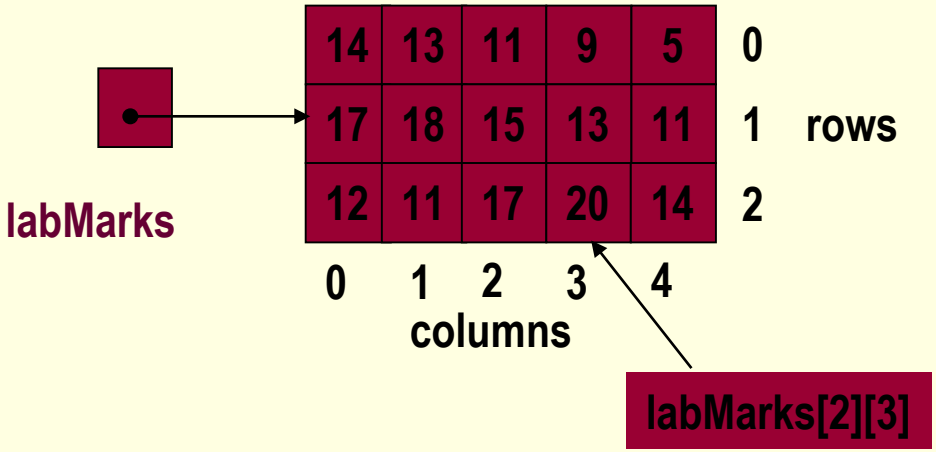
Based on Slides of Dr. Norazah Yusof

Two-dimensional Arrays

- Arrays can be extended to several dimensions (Two or more).
- A two dimensional array, for example, is a repetition of one-dimensional arrays (i.e. array of arrays).

```
int[][] labMarks = {
                {14, 13, 11, 9, 5},
                {17, 18, 15, 13, 11},
                {12, 11, 17, 20, 14}
              }
```

array of three rows and five columns



labMarks is a two dimensional array of marks of three students in five lab tests. Student numbers are represented along rows from 0 to 2. Lab tests are numbered along five columns. Each array cell contains student's marks.

Declaration of Two-dimensional Arrays

```
// Declare array ref var
dataType[][] refVar;
// Create array and assign its reference to variable
refVar = new dataType[5][5];

            or

// Combine declaration and creation in one statement
dataType[][] refVar = new dataType[5][5];
// Alternative syntax
dataType refVar[][] = new dataType[5][5];
```

Declaring Variables of Two-dimensional Arrays and Creating Two-dimensional Arrays

Example:

```
int[][] matrix = new int[5][5];
```

or

```
int matrix[][] = new int[5][5];
```

Accessing Two-Dimensional Array

```
for (int i = 0; i < matrix.length; i++)
  for (int j = 0; j < matrix[i].length; j++)
    matrix[i][j] = (int)(Math.random() * 100);
```

Two-dimensional Array Illustration

	0	1	2	3	4
0					
1					
2					
3					
4					

```
matrix = new int[5][5];
```

	0	1	2	3	4
0					
1					
2		7			
3					
4					

```
matrix[2][1] = 7;
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9
3	10	11	12

```
int[][] array = {
   {1, 2, 3},
   {4, 5, 6},
   {7, 8, 9},
   {10, 11, 12}
};
```

matrix.length? 5

matrix[0].length? 5

array.length? 4

array[0].length? 3

Declaring, Creating, and Initializing Using Shorthand Notations

You can also use an array initializer to declare, create and initialize a two-dimensional array. For example,

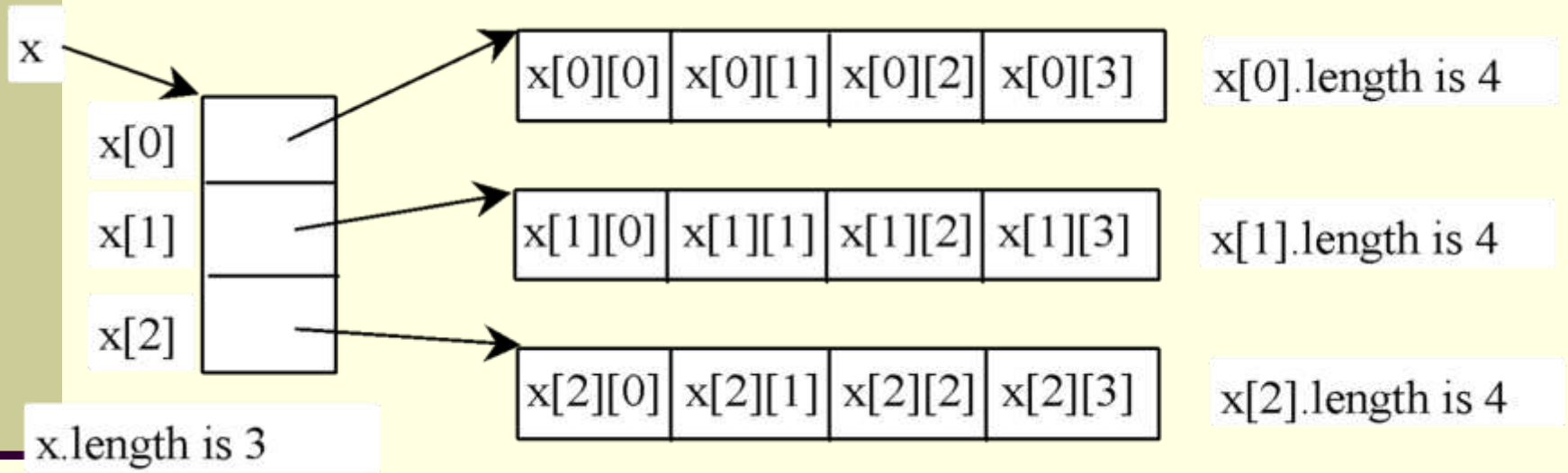
```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

Same as

```
int[][] array = new int[4][3];
array[0][0] = 1; array[0][1] = 2; array[0][2] = 3;
array[1][0] = 4; array[1][1] = 5; array[1][2] = 6;
array[2][0] = 7; array[2][1] = 8; array[2][2] = 9;
array[3][0] = 10; array[3][1] = 11; array[3][2] = 12;
```

Lengths of Two-dimensional Arrays

int[][] x = new int[3][4];



Lengths of Two-dimensional Arrays, cont.

```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

array.length

array[0].length

array[1].length

array[2].length

array[3].length

array[4].length ArrayIndexOutOfBoundsException

Processing Two-Dimensional Arrays

- Declare a two-dimensional array named matrix of type integer with 3 rows and 4 columns:

```
int[][] matrix = new int[3][4];
```

- Some processing examples:

1. Input arrays with values read from the keyboard.
2. Printing arrays.
3. Summing all elements.
4. Summing elements by column
5. Finding which row has the largest sum.

Accessing Two-Dimensional Array Elements

- **Programs that process two-dimensional arrays can do so with nested loops.**
- **To fill the matrix array:**

```
for (int row = 0; row < matrix.length; row++)
{

  for (int col = 0; col < matrix[row].length; col++)
  {
    System.out.print("Enter a value: ");
    matrix[row][col] = keyboard.nextInt();
  }
}
```

Number of rows, not the largest subscript

Number of columns, not the largest subscript

`keyboard` references a `Scanner` object

Accessing Two-Dimensional Array Elements

- **To print out the `matrix` array:**

```
for (int row = 0; row < matrix.length;
  row++)
{
   for (int col = 0; col <
  matrix[row].length; col++)
  {
     System.out.println(matrix[row][col]);
   }
}
```

Summing The Elements of a Two-Dimensional Array

```
int[][] numbers = { { 1, 2, 3, 4 },
                    {5, 6, 7, 8},
                    {9, 10, 11, 12} };
int total;
total = 0;
for (int row = 0; row < numbers.length; row++)
{
  for (int col = 0; col < numbers[row].length;
  col++)
    total += numbers[row][col];
}
System.out.println("The total is " + total);
```

Ragged Arrays

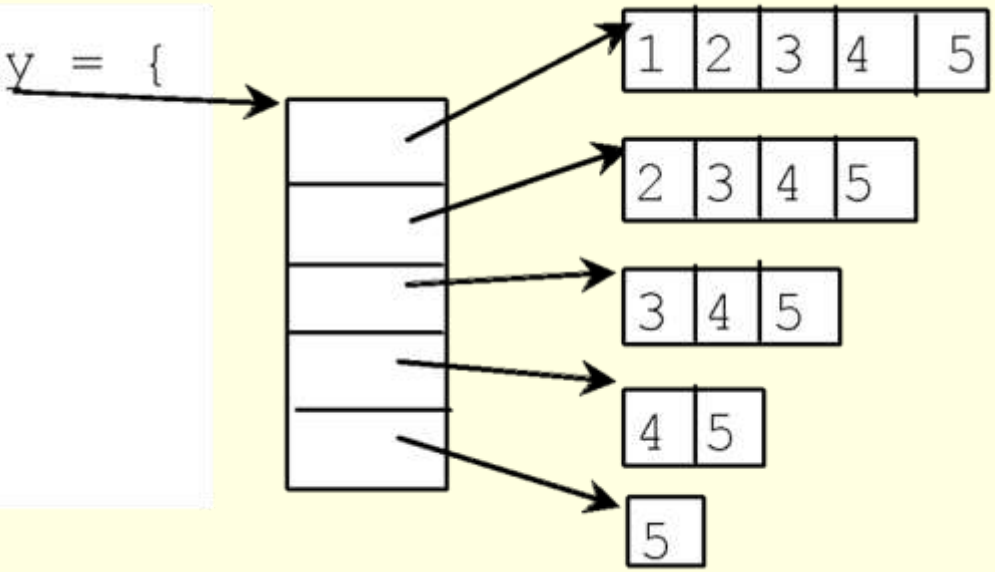
Each row in a two-dimensional array is itself an array. So, the rows can have different lengths. Such an array is known as *a ragged array*. For example,

```
int[][] matrix = {
  {1, 2, 3, 4, 5},
  {2, 3, 4, 5},
  {3, 4, 5},
  {4, 5},
  {5}
};
```

```
matrix.length is 5
matrix[0].length is 5
matrix[1].length is 4
matrix[2].length is 3
matrix[3].length is 2
matrix[4].length is 1
```

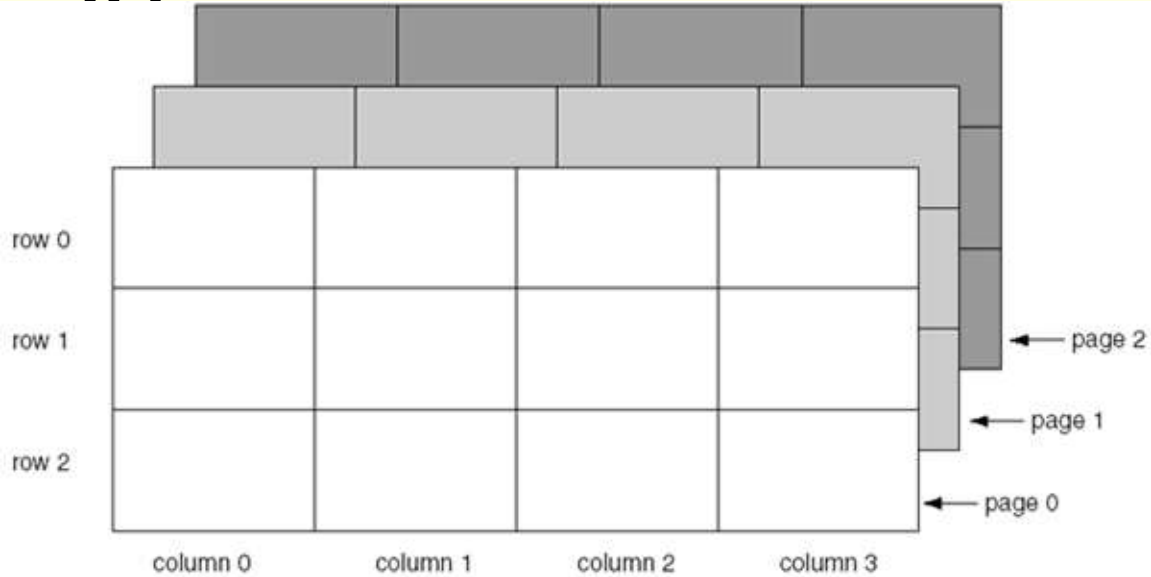
Ragged Arrays, cont.

```
int[][] triangleArray = {
   {1, 2, 3, 4, 5},
   {2, 3, 4, 5},
   {3, 4, 5},
   {4, 5},
   {5}
};
```



More Than Two Dimensions

- **Java does not limit the number of dimensions that an array may be.**
- **More than three dimensions is hard to visualize, but can be useful in some programming problems.**



Using the `Arrays` Class

- **`Arrays` class**
  - **Contains many useful methods for manipulating arrays**
  - **`static` methods**
    - **Use with class name**
    - **Without instantiating `Arrays` object**
  - **`binarySearch()` method**
    - **Convenient way to search through sorted lists of values of various data types**
    - **List must be in order**

Useful Methods of the `Arrays` Class

Method	Purpose
`static int binarySearch(type [] a, type key)`	Searches the specified array for the specified key value using the binary search algorithm
`static boolean equals(type[] a, type[] a2)`	Returns `true` if the two specified arrays of the same type are equal to one another
`static void fill (type[] a, type val)`	Assigns the specified value to each element of the specified array
`static void sort (type[] a)`	Sorts the specified array into ascending numerical order
`static void sort(type[] a, int fromIndex, int toIndex)`	Sorts the specified range of the specified array into ascending numerical order

Table 8-3 Useful methods of the `Arrays` class

Exercises

- **Write two java program to create and print this array:**

  1 2 3 4 5

  6 7 8 9 10

  11 12 13 14 15

  16 17 18 19 20

1. **Using one statement to declare, create and initialize the array and nested for loop to print the output**
2. **Using Scanner class to get the input values from the user to initialize the array and nested for loop to print the output**

```
public class Array {
        public static void main(String[] args)   {
            int[][] a = {{1, 2, 3, 4, 5},
                         {6, 7, 8, 9, 10},
                         {11, 12, 13, 14, 15},
                         {16, 17, 18, 19, 20}};

                        for(int i=0; i<a.length; i++) {
                              for(int j=0; j<a.length;j++) {
                                 System.out.print(a[i][j]+" ");
                                    }
        System.out.println("");
          }
   }
}
```